

GoF Design Pattern Quick Reference

How to use this

Start with **Section 1** — work top to bottom through the signals until one matches what you're looking at. Most classes will match zero or one signal; that's normal, not every class needs a pattern. Use **Section 2** as a lookup once you know a pattern's name and need the definition.

Why the three categories aren't just a filing system

Creational and Behavioral patterns are almost always about a **decision or event happening while the program runs** — deciding which subtype to build, switching behavior by status, notifying listeners, intercepting a call. The test for whether you need one is usually: *does this decision get made repeatedly, at runtime, from more than one place?*

Structural patterns are different — they're more often a **fixed relationship between classes that's simply true once you've designed it that way**, not something that "happens." A Composite tree (folders containing files or more folders) has that shape whether or not the program is currently running. A Bridge between an abstraction and its implementation is a standing fact about the class design, not an event. Don't apply the "does this happen at runtime, repeatedly, from multiple places?" test to a Structural pattern — it's the wrong question for that category, even though it's exactly the right question for Creational and Behavioral ones.

Section 1: "What am I looking at?" → Pattern

Creational — I'm creating/constructing something

If you see this...	Consider
You need exactly one shared, coordinating instance system-wide	Singleton
You're choosing which concrete subtype to build based on input, avoiding a big if/else of new calls	Factory Method
You need to create a whole <i>family</i> of related objects that must be used together (e.g., all "dark mode" UI pieces)	Abstract Factory
Construction has many optional or ordered steps, or the constructor would need a huge parameter list	Builder

Creating a new object is more expensive than copying an existing one

Prototype

Structural — I'm arranging how classes relate/connect

If you see this...	Consider
You're wrapping a 3rd-party or legacy system behind your own interface	Adapter
You have two independent dimensions of variation and don't want a subclass for every combination (e.g., shape type × render engine)	Bridge
You have a tree structure (folders/files, UI components) and want to treat single items and groups the same way	Composite
You want to add features to one specific object at runtime, without subclassing for every combination of features	Decorator
Many subsystems need one simple front door for clients to call	Facade
You have huge numbers of similar objects and want to share common state to save memory	Flyweight
You need to guard, control, or lazy-load access to something (security check, remote resource, expensive object)	Proxy

Behavioral — I'm coordinating how objects communicate/behave

If you see this...	Consider
A request might be handled by one of several objects, and the sender shouldn't need to know which	Chain of Responsibility
You need to queue, log, or undo an action — an operation needs to be treated as an object	Command
You need to interpret sentences in a small custom language/expression grammar (rare in typical apps)	Interpreter
You need to step through a collection without exposing its internal structure	Iterator
Many objects talk to each other in a tangled many-to-many way; you want to centralize that communication	Mediator
You need undo/rollback/backup-restore without exposing an	Memento

object's internals	
One object's state change needs to notify several unrelated listeners	Observer
An object's behavior changes based on its own current status/lifecycle stage (lots of conditionals on "current state")	State
Several interchangeable ways to do the same job, chosen at runtime	Strategy
Several classes follow the same overall algorithm but differ in a few specific steps	Template Method
You need to add new operations to a stable set of classes without modifying those classes (rare, adds real complexity)	Visitor

Section 2: Full list by category

Creational (5)

Pattern	One-line definition
Singleton	Ensures a class has only one instance, with a global access point to it
Factory Method	Defines an interface for creating an object, lets subclasses decide the concrete type
Abstract Factory	Creates families of related objects without specifying their concrete classes
Builder	Separates the construction of a complex object from its final representation
Prototype	Creates new objects by cloning an existing instance

Structural (7)

Pattern	One-line definition
Adapter	Converts one class's interface into another interface clients expect
Bridge	Decouples an abstraction from its implementation so both can vary independently
Composite	Composes objects into tree structures; treats individual items and groups uniformly
Decorator	Attaches new responsibilities to an object dynamically, as an alternative to subclassing
Facade	Provides a simplified, single interface to a complex subsystem

Flyweight	Shares common state across many fine-grained objects to reduce memory use
Proxy	Provides a surrogate/placeholder that controls access to another object

Behavioral (11)

Pattern	One-line definition
Chain of Responsibility	Passes a request along a chain of handlers until one handles it
Command	Encapsulates a request as an object, enabling queuing, logging, and undo
Interpreter	Given a language, defines a representation of its grammar plus an interpreter
Iterator	Provides sequential access to a collection's elements without exposing its structure
Mediator	Centralizes how a set of objects interact, reducing direct references between them
Memento	Captures and externalizes an object's internal state for later restoration
Observer	Defines a one-to-many dependency: when one object changes, all dependents are notified
State	Lets an object alter its behavior when its internal state changes
Strategy	Defines a family of interchangeable algorithms and makes them swappable at runtime
Template Method	Defines an algorithm's skeleton, deferring specific steps to subclasses
Visitor	Lets you define a new operation on a class hierarchy without modifying the classes

Section 3: Plain-language analogies

Use this when a pattern's formal definition isn't clicking. Each one is an everyday situation with the same underlying shape as the pattern — no code, no UML.

Creational

Pattern	Everyday version
Singleton	A school's front office — one office, everyone goes to that same one Ordering at a coffee counter —

Factory Method	you name the type, the barista decides how to make it
Abstract Factory	Buying a matched furniture set — one choice (“Scandinavian style”) produces a coordinated chair, table, and lamp together
Builder	Building a sandwich at Subway — fixed steps (bread, meat, toppings, sauce) ending in one finished product
Prototype	Photocopying a filled-out form instead of starting from blank — copy something existing and tweak what differs

Structural

Pattern	Everyday version
Adapter	A travel plug adapter — your charger’s plug never changes; the adapter translates it to the local socket
Bridge	A TV remote and the TV it controls — the remote’s buttons work the same regardless of which TV brand is on the other end
Composite	Nested folders on a computer — “what’s the total size?” works the same whether it’s one file or a whole nested tree
Decorator	Customizing a coffee order — each add-on (extra shot, whipped cream) wraps the base drink without needing a new menu item per combination
Facade	A car’s start button — hides the ignition, fuel system, and electronics working together underneath
Flyweight	A library sharing one copy of a popular book across many borrowers, instead of printing a personal copy for each
Proxy	A security guard at a building entrance — checks your ID before letting you reach the office you’re visiting

Behavioral

Pattern	Everyday version
Chain of Responsibility	A customer service call escalating from tier 1 to tier 2 to tier 3 — you never know in advance who’ll actually solve it
Command	A restaurant order ticket — a slip of paper that can be queued, handed to any cook, or pulled back to cancel

Interpreter	A calculator reading “ $3 + 4 \times 2$ ” and evaluating it by grammar rules — rare outside custom small languages
Iterator	Flipping through a photo album one page at a time — you don’t need to know how the photos are actually stored
Mediator	An air traffic control tower — planes coordinate through the tower instead of talking to each other directly
Memento	A video game save point — save progress, reload it later, without the game exposing how it’s stored internally
Observer	Subscribing to a YouTube channel — the creator posts once, every subscriber is notified automatically
State	A traffic light — same object, but what it does changes completely depending on whether it’s red, yellow, or green
Strategy	Choosing a route in Google Maps — walking, driving, biking are interchangeable methods for the same goal
Template Method	A cake recipe — preheat, mix, bake, cool stays fixed; only the mix-in step changes between chocolate and vanilla
Visitor	A tax auditor visiting different business types — the auditor adapts to each type without the businesses changing for it

The test that matters more than the table

Before committing to any pattern, ask: **does this class/relationship already show the signal, or am I adding the pattern because it exists?** A pattern is the right call when it solves a problem you can point to in your requirements. If you can’t say *which* user story or design problem justifies it, that’s a sign to leave it out — this is the same YAGNI check you’ve been applying throughout the bookstore project.